

Inferring Numerical Abstract Domain Types from Concrete Program States

Kenny Ballou[✉]
California State University San
Marcos
San Marcos, USA
kballou@csusm.edu

Teddy Moore
Boise State University
Boise, USA
tedmoore@u.boisestate.edu

Elena Sherman
Boise State University
Boise, USA
elenasherman@boisestate.edu

Abstract

The effectiveness of interpretation-based numerical analyses depends on the choice of abstract domain. Domains such as Zones and Octagons differ in expressiveness and cost, so the goal is to identify the least expressive domain that is sufficient for a given analysis context. The challenge is that this choice varies across variables and program locations, while obtaining variable-location domain recommendations using lightweight static techniques can be difficult.

This paper proposes an approach for inferring the recommended numerical domain type for each variable at each program location from concrete program states. We use MultisetGA to generate test suites that yield large, strategically distributed sets of unique concrete states, with at least 1,000 states per location. We then use AbsDaikon, an extension of Daikon, to infer invariants matching numerical abstract-domain templates. These templated invariants are analyzed to identify the domain type recommended for each variable. Our preliminary results show that, over all variable-location pairs, the inferred recommendations exactly match the computed reference domains in 85.6% of cases, are more expressive in 14.4% of cases, and are never less expressive.

CCS Concepts: • **Theory of computation** → *Program analysis; Program verification; Invariants.*

Keywords: abstract interpretation, numerical abstract domains, invariant inference, abstraction type selection

ACM Reference Format:

Kenny Ballou, Teddy Moore, and Elena Sherman. 2026. Inferring Numerical Abstract Domain Types from Concrete Program States. In *Proceedings of the 11th ACM SIGPLAN International Workshop on Numerical and Symbolic Abstract Domains (NSAD '26)*, October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3840563.3843707>



This work is licensed under a Creative Commons Attribution 4.0 International License.

NSAD '26, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2933-1/2026/10

<https://doi.org/10.1145/3840563.3843707>

1 Introduction

To be effective and efficient, static analysis needs to know at what level of abstraction to analyze a program. If an abstraction is too coarse, an analysis may compute insufficient information, even though it does so efficiently. On the other hand, if an abstraction is too fine, the analysis may run out of resources before completing its task. Thus, an important research direction is to automatically determine an appropriate level of abstraction for a given program [1, 5, 8, 11, 18, 19]. This choice is further complicated by the fact that the optimal abstraction is often not uniform across an entire program, but varies by program context. That is, only certain parts of the program may need to be analyzed using a finer abstraction, while the remaining parts can be analyzed using a coarser one. For example, the value of k for object-sensitivity levels can be set differently for different objects [11, 19].

An important abstraction choice within abstract interpretation is the numerical abstract domain [4] used by the analysis. In previous work, Apel et al. select which domain type, among two incomparable abstract-domain types, should be assigned to each variable [1]. This eliminates the need to maintain several abstract states for each variable while preserving the same level of precision. Their decision is based on a lightweight static analysis of variable usage. However, in that work the choice of abstract domain is fixed for the entire analysis, so the abstraction context is per program variable.

When considering numerical abstract domains such as Zones [12], and Octagons [13], identifying the appropriate abstraction context using only lightweight static analysis may be more challenging. Moreover, the required type of numerical abstract domain may change from one program location to another. For example, a variable may initially have a constant value and later be incremented, requiring an Interval domain. In this scenario, the abstraction context should be a variable-location pair rather than a variable alone. Accurately computing such fine-grained abstraction information statically may be difficult and could require running the analysis with the most precise numerical domain to obtain the needed information, which defeats the purpose of precomputing this information before analysis.

This work presents an exploratory study of using concrete states to infer sufficiently expressive abstract-domain types

for each variable-location pair. The concrete states are collected from program executions on a specially designed test suite. Unlike previous work on invariant inference from concrete states, such as Daikon [6] or DIG [16], our work infers the domain type, e.g., Intervals or Zones. First, we use the MultisetGA [14] tool, which extends Evosuite [7], to generate a test suite that produces a large number of uniformly distributed, unique concrete states for a given program location. We then extend Daikon to create AbsDaikon, which infers invariants for specific abstract-domain templates, namely those matching Octagon, Zone, or Interval forms. Finally, the inferred templated invariants are analyzed to identify the corresponding domain type.

The main difference from the work on invariant inference is that our approach does not aim to infer the result of static analysis, which would be inherently unsound from concrete executions alone. Instead, we use inferred invariants to determine the type of abstract domain required for each variable at a given program location. We consider five domains with the following expressiveness ordering: Octagon (O) $\succ$ Zone (Z) $\succ$ Interval (I) $\succ$ Constant (C) $\succ$ Unbounded $\top$. We compare our inferred domain types with those computed by a static approach. Our empirical evaluation shows that, over variable-location pairs, the proposed approach agrees with the computed domain type in approximately 85% of cases. In the remaining cases, the inferred recommendation is more expressive than the computed domain. Most importantly, the inferred domain types never recommend a less expressive domain than the computed domain.

The rest of the paper is organized as follows. In the next section, Section 2, we present a motivating example and highlight our approach. We then, in Section 3, describe AbsDaikon, our extension of Daikon for inferring invariants over numerical abstract-domain templates. This is followed by a description of how domain types are identified from those inferred invariants. In Section 4, we evaluate our approach on benchmark programs. We conclude with future directions for this exploratory study.

2 Motivating Example

In this section, we use a small example to illustrate why numerical abstract-domain selection should be both variable- and location-sensitive. The example shows that the least expressive, sufficient domain can change after individual statements and can differ across variables at the same program location. We first show the desired domain-type annotations, then explain how the same information can be obtained from a precise Octagon analysis. Finally, we motivate our alternative approach: inferring this information from generated concrete states before configuring a static analyzer.

Variable-location-level Abstraction. To explain variable-location-level abstraction, consider the example in Listing 1. The annotations before and after each statement describe the

Listing 1. Location-sensitive inferred domains.

```

1 int example(int x, int y) {
2   int z;
3   // Top: x,y,z
4   if (y >= 0) {
5     // Interval: y; Top: x,z
6     z = y + 2;
7     // Zone: y,z; Top: x
8   } else if (x <= y + 1) {
9     // Zone: x,y; Top: z
10    z = x - 1;
11    // Zone: x,y,z
12  } else {
13    // Zone: x,y; Top: z
14    z = 4 - y;
15    // Octagon: x,y,z
16  }
17  // Top: x,y,z
18  return z;
}
```

best numerical domain abstraction for each variable. In this example, on the true branch of the first conditional statement, $y \geq 0$, the variable y is adequately expressed in the Interval domain while the other two variables remain unbounded.

After $z = y + 2$, both y and z require the Zones domain to encode their relation, while x remains unbounded. Interestingly, despite these relations at the end of each branch, the resulting state before the return statement is unbounded, as no common useful Octagonal bounds remain. This indicates that, at the join point, the octagonal abstraction cannot express any nontrivial relations among the three variables. The rest of the best-abstraction annotations indicate that only after $z = 4 - y$ does an Octagonal state provide the best abstraction for all three variables. Most of the other abstract states require Zone abstraction.

Domain Types from Static Analysis. To obtain an accurate variable-location context for the abstraction level, one needs to run a static analysis with the most precise domain, in our case using Octagons. We then pre-process each abstract state by first removing spurious constraints and then examine the resulting state to determine the type of relations between variables. Listing 2 shows the same code annotated with the results of this analysis, where an Octagonal abstract state is expressed as a conjunction of predicates over the program variables. If a variable does not appear in any constraint, it is unbounded.

To highlight the importance of removing spurious constraints, consider the statement $z = y + 2$. The first formula, labeled “raw,” is a direct translation from the state encoding, which is represented by a $2N \times 2N$ matrix, where $N = |V|$ is the number of variables in the program under analysis; in this case, $N = 3$. Figure 1 shows the matrix representation of this Octagonal abstract state.

To properly merge flows, a numerical abstract state is converted into a canonical form by performing a closure operation, which introduces all constraints that can be inferred from the state. As a result, the “raw” constraints now contain

$$\begin{array}{c}
 y^+ \quad y^- \quad z^+ \quad z^- \\
 \begin{array}{c}
 y^+ \\
 y^- \\
 z^+ \\
 z^-
 \end{array}
 \begin{bmatrix}
 0 & +\infty & -2 & +\infty \\
 0 & 0 & +\infty & +\infty \\
 2 & +\infty & 0 & +\infty \\
 2 & +\infty & -4 & 0
 \end{bmatrix}
 \end{array}$$

Figure 1. Octagonal matrix for $y \geq 0 \wedge z \geq 2 \wedge y \leq z - 2 \wedge z \leq y + 2 \wedge z \geq -2 - y$.

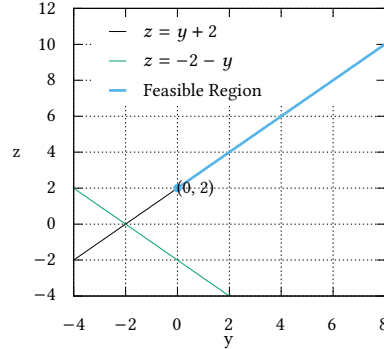


Figure 2. Example of a spurious constraint removed by $w0z$ reduction

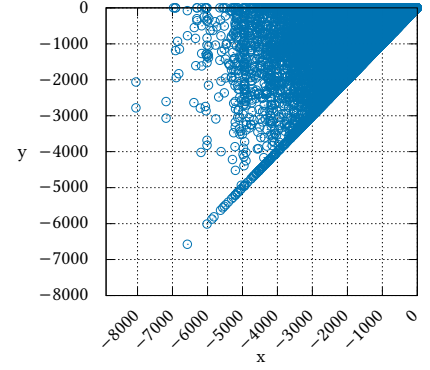


Figure 3. Data point distribution for (x, y) variables

Listing 2. Simplifying inferred constraints.

```

1  int example(int x, int y) {
2      int z;
3      // true
4      if (y >= 0) {
5          // y >= 0
6          z = y + 2;
7          // raw:  y >= 0 && z >= 2 && y <= z-2 &&
8          // z <= y+2 && z >= -2-y
9          // red:  y >= 0 && z >= 2 && y <= z-2 &&
10         // z <= y+2
11         // simpl: y >= 0 && z = y+2
12     } else if (x <= y + 1) {
13         // y <= -1 && x <= y+1
14         z = x - 1;
15         // y <= -1 && x <= y+1 && z = x-1
16     } else {
17         // y <= -1 && x >= y+2
18         z = 4 - y;
19         // y <= -1 && x >= y+2 && z = 4-y
20     }
21     // true
22     return z;
    }
```

the spurious constraint $z \geq -2 - y$, which is of Octagonal type. Figure 2 shows that this constraint lies in the infeasible region of the x, y solution space. Clearly, such spurious constraints should be eliminated using $w0z$ reduction [3] before categorizing an abstract state into domain types.

The constraint labeled “red” is the resulting formula after applying $w0z$ reduction, i.e., without spurious constraints. However, this constraint still contains the redundant predicate $z \geq 2$, which can be inferred from the remaining predicates. Such redundant constraints can be eliminated with Larsen reduction [10]. The constraint annotated with “simpl” is the resulting abstract state. The rest of the abstract states in Listing 2 are shown in their reduced and simplified forms. **Domain Types.** To determine a variable’s best abstraction type, we can analyze the matrix encoding of the abstract state. If the entry for x and $\bar{y}$, or conversely the entry for $\bar{x}$ and y , has a non-top value, then the state contains an

Octagonal constraint and therefore requires an Octagonal state to express this relation. However, if those entries are not present, we can examine the entries for x and y , or y and x , to identify Zonal constraints. Similar reasoning over the matrix encoding is used to identify Intervals and Constants.

Without $w0z$ reduction, the abstract domain type for y and z after line 6 in Listing 2 would be identified as Octagonal instead of Zonal. However, Larsen reduction does not change the type, since it only removes constraints that can be inferred from existing constraints. Clearly, a finer abstraction cannot be inferred from coarser constraints alone. Thus, if we apply only Larsen reduction to the first constraint, it removes $z \geq 2$, but not the spurious constraint $z \geq -2 - y$.

After querying each Octagonal state in Listing 2, we obtain the same categorization of domain types for each variable as in Listing 1.

Such variable-location abstraction levels have limited utility after an entire analysis has already been run with the most precise abstract domain. To take advantage of this optimal abstraction-level information, we need to obtain it by different means and then configure a static analyzer with those abstraction levels. One option is to perform a lightweight static analysis similar to Apel et al. [1]; however, this approach might only provide an upper precision bound for a given variable and might not necessarily track how the required abstraction changes from statement to statement.

Domain Types from Concrete States. This work offers an alternative approach to obtaining abstraction-level information by inferring it from concrete program states, which we obtain by executing the program on thousands of test cases. This dynamic inference approach has previously been applied to infer program invariants [9, 15, 17] using inference engines such as Daikon [6] or DIG [16]. Unlike those techniques, our approach produces constraints only for templates that describe numerical abstract domains. In Section 3.1, we explain how we implemented AbsDaikon, an extension of

the Daikon invariant inference tool that produces invariants corresponding to numerical abstract domains.

As with any dynamic technique, confidence in the obtained invariants depends on the number and quality of the sampled concrete states. In this work, we use MultisetGA [14], implemented in Evosuite [7], which can generate thousands of test cases such that the targeted location has many unique and well-distributed concrete states. The inferred Octagonal constraints from concrete states are presented in Listing 3. One can see that they also contain spurious and redundant predicates, which our domain-type classification algorithm handles appropriately. For this example, the inferred domain types per program variable in Listing 3 match the computed domain types from Listing 2, which indicates the feasibility of the proposed approach.

Note, inferring domain types is less restrictive than inferring invariants. Thus, an inferred predicate $x \geq 2$ might under-approximate the actual predicate $x \geq 0$, which is a common issue in dynamic invariant inference. However, both predicates have the same Interval domain type.

With thousands of concrete states used for inference, the confidence in identifying the correct domain type is relatively high. The plot in Figure 3 demonstrates the distribution of points for the $z = x - 1$ line projected onto the x and y variables. As one can see, the data points outline the shape of a Zonal domain, which AbsDaikon reports as an inferred constraint that contains spurious and redundant constraints.

In the next section, we describe our overall approach, in particular the AbsDaikon algorithm. This is followed by a description of domain-type identification and the domain-type identification algorithm that we briefly introduced here.

3 Approach

3.1 Inference

AbsDaikon is a modification of Daikon [6] that infers constraints, for each instrumented location, directly from the concrete states generated by MultisetGA [14]. AbsDaikon disables Daikon’s default invariants and enables only the templates that represent the constraints for each domain of interest. In this case, Intervals (which includes Constant, e.g., singleton intervals), Zones, and Octagons.

Daikon already infers the unary bounds $l \leq x \leq u$ through its LowerBound and UpperBound invariants, which supply the Interval constraints. Using concrete states Daikon computes l and u constants, inserts into the templates and output them as inferred invariants. The relational constraints that distinguish Zones and Octagons, however, do not exist in Daikon. As such, we added unit binary invariant templates to cover these additional numerical domains.

Each template within Daikon reports the tightest template constant consistent with every observed concrete state to ensure an inferred invariant indeed tightly bounds all observed concrete states. Daikon accomplishes this by maintaining

Listing 3. Inferred constraints before simplification.

```

1  int example(int x, int y) {
2      int z;
3      if (y >= 0) {
4          // y >= 0
5          z = y + 2;
6          // y >= 0 && z >= 2 && -z-y <= -2 &&
7          // y-z <= -2 && z-x <= 2
8      } else if (x <= y + 1) {
9          // x <= 0 && y <= -1 && x-y <= 1 &&
10         // y+x <= -1
11         z = x - 1;
12         // x <= 0 && y <= -1 && z <= -1 &&
13         // x-y <= 1 && y+x <= -1 && x-z <= 1 &&
14         // z+x <= -1 && z-x <= -1 && z+y <= -2 &&
15         // z-y <= 0
16     } else {
17         // y <= -1 && y-x <= -2
18         z = 4-y;
19         // y <= -1 && y-x <= -2 && -z-x <= -6 &&
20         // -z-y <= -4 && y-z <= -6 && z+y <= 4
21     }
22     return z;
}

```

the maximum of its linear expression across all data points. For example, if the constraint is $x - y \leq c$, it is tracked with

$$c = \max_{\sigma \in S} (\sigma(x) - \sigma(y)),$$

where S is the set of states observed at the location. Because c is the observed maximum, the constraint is the tightest bound which no observed state violates. Because no observation can falsify such a bound, whether it is reported depends on a confidence measure.

A relational constraint is only meaningful if it constrains a pair beyond what the variables’ own ranges already imply. AbsDaikon, therefore, computes confidence for each relational template by its tightness relative to the interval. For the constraint $x - y \leq c$, let

$$B = \max_{\sigma \in S} \sigma(x) - \min_{\sigma \in S} \sigma(y)$$

be the largest difference between x and y independent of all states. The *confidence* is then the fraction by which the observed constant c improves on this bound:

$$\text{conf}(x - y \leq c) = \max\left(0, \min\left(1, \frac{B - c}{B}\right)\right).$$

A constraint that merely restates the box ($c = B$) scores 0, whereas one far tighter than the box ($c < B$) approaches 1. This same formula applies to each template, where the variable extremes determine the value of B . Daikon supports configuring the confidence threshold τ to adjust the reported constraints. Lowering the confidence level accepts constraints closer to the interval bounds.

Consider a location recording two variables x, y over the four states in Table 1. Algorithm 1 feeds each state to every template type, lines 1–5. After collecting the confidence

Algorithm 1 AbsDaikon Inference.

Require: States S ; Templates T ; Threshold τ
Ensure: Inferred predicate set P

```

1: for all  $\sigma \in S$  do
2:   for all  $t \in T$  do
3:      $t.ADD(\sigma)$  //Each state is given to each template
4:   end for
5: end for
6:  $P \leftarrow \emptyset$ 
7: for all  $t \in T$  do
8:   if  $t.CONF() \geq \tau$  then
9:      $P \leftarrow P \cup \{t.PRED()\}$ . // Add predicate if confident
10:  end if
11: end for
12: return  $P$ 
    
```

thresholds for each variable, Algorithm 1 decides what predicate should be classified as inferred ones, lines 7–11.

An interval template records a variable's extremes and a relational template keeps the running maximum of its expression, its tightest constant c . After the pass, the extremes are $lo[x]=-4$, $hi[x]=3$, $lo[y]=-5$, $hi[y]=2$, and each relational template holds its tightest constant c .

Table 1. Observed states S .

σ	x	y
σ_1	-4	-5
σ_2	1	0
σ_3	3	2
σ_4	-2	-3

For the difference $x - y$, the expression equals 1 in every state, so $c = 1$; its box bound is $B = hi[x] - lo[y] = 3 - (-5) = 8$, giving $conf = (8 - 1)/8 = 0.88$. The opposite direction $y - x$ gives $c = -1$ with $B = hi[y] - lo[x] = 2 - (-4) = 6$ and $conf = \min(1, (6 - (-1))/6) = 1$. The two together pin $x - y = 1$. For the sum $x + y$ the maximum is $c = 5$, but the box bound is also $B = hi[x] + hi[y] = 5$, so $conf = 0$; likewise $-x - y$ has $c = B = 9$. Both octagonal templates merely restate the interval box and are filtered out. The inferred predicate set is, therefore:

$$P = \{-4 \leq x \leq 3, -5 \leq y \leq 2, x - y \leq 1, y - x \leq -1\},$$

Thus, the pair return Zone template predicate rather than an Octagon one.

3.2 Identifying Domain Type

To identify the domain types for each variable, we require an algorithm which can determine the minimal invariant kinds for each variable. That is, given a set of invariants, we need to identify the minimal domain necessary for expressing each variable. For example, given the invariant $y \geq 0$, we can

Algorithm 2 Algorithm for mapping variables to domain types.

Require: M , a W0Z reduced Octagon Matrix

Ensure: $\forall v \in M : DT(v) \in \{C \prec I \prec Z \prec O\}$

```

1:  $N \leftarrow \text{sizeof}(M)$ 
2: for all  $i \in \{x + 2 | 0 \leq x \leq N\}$  do
3:    $DT(v) \leftarrow \top$ 
4: end for
5: for all  $i \in \{x + 2 | 0 \leq x \leq N\}$  do
6:   if  $M_{ii} \equiv -1 \times M_{ii}$  then
7:      $DT(v) \leftarrow C$ 
8:   else if  $M_{ii} \neq \top \vee M_{ii} \neq \top$  then
9:      $DT(v) \leftarrow I$ 
10:  end if
11: end for
12: for all  $i \in \{x + 2 | 0 \leq x \leq N\}$  do
13:   for all  $j \in \{x + 2 | i \leq x \leq N\}$  do
14:      $\bar{i} \leftarrow i \oplus 1$ 
15:      $\bar{j} \leftarrow j \oplus 1$ 
16:     if  $(M_{ij} \neq \top \vee M_{ji} \neq \top) \wedge DT(v) \prec Z$  then
17:        $DT(v) \leftarrow Z$ 
18:     end if
19:     if  $M_{ij} \neq \top \vee M_{ij} \neq \top$  then
20:        $DT(v) \leftarrow O$ 
21:     end if
22:   end for
23: end for
24: return  $DT$ 
    
```

deduce that y is fully representable by the interval domain. However, given the invariant $y \geq 0 \wedge y - x \leq -2 \wedge x - y \leq 2$, the zones domain sufficiently encodes x and y .

Since we maximally support identifying Octagon domains, and the Octagon domain contains Zones, Intervals, and Constants as sub-expression types, we use the Octagon domain, more specifically, we use its difference bounded matrix (DBM) representation to decide the domain types for each variable.

Algorithm 2 provides the pseudocode for our domain type assignment algorithm. It maps every variable within an Octagon state to a domain type: either $\top$ for unbounded or unknown variables, C for the *constant* domain, I for the *interval* domain, Z for the *zone* domain, or O for the *octagon* domain. Algorithm 2 requires a reduced Octagon, specifically, one using the *w0z* reduction algorithm [3] such that spurious constraints are removed, yet, leaving interval valued constraints. Initially, all variables are initialized to the *unbounded* category, lines 2–4. Next, on lines 5–11, the algorithm loops over all variables, initializing their initial domain type to either *constant* provided both bounds are equivalent, line 6, or *interval* otherwise. Then, on lines 12–23, the algorithm checks for relational constraints consistent with the *Zone* domain or *Octagon* domain. Concretely, on line 16, the algorithm checks if the current variable has unit difference

constraints to other variables and if it has only been classified as constant or interval. Similarly, line 19 checks if the current variable has unit sums constraints between other variables. These relationships promote the current variable to the *Octagon* domain type. Finally, we return our variable to domain type map.

As an optimization, we need only consider the even values of i and j , as these correspond to the *positive* pair of variables within the Octagon encoding of the constraints. Moreover, since an Octagon encodes equivalent constraints in the upper and lower triangle of its DBM, we need only consider one of the triangles to completely test all relations. Thus, Algorithm 2 has a quadratic runtime complexity, that is, $O(N^2)$, where N is the number of program variables.

4 Experiments and Results

4.1 Experiment set-up

We evaluate whether domain types inferred from concrete states match those computed by static analysis. It covers 20 benchmark programs and 1,003 variable-location pairs. For every variable at every instrumented location, we compare the inferred domain type against the static analysis results.

Benchmarks. Benchmarks come from two main sources. The first is a set of established equivalence-checking benchmarks, REVE and CLEVER [2]. The second source, *shift-zone*, is a set we created specifically to exercise domain changes. Each program is small, integer-only, and specifically crafted to incur domain changes in variables between locations.

Test generation and inference. For each instrumented location, we use MultisetGA [14] to generate a suite yielding at least 1,000 unique, well-distributed concrete states. We instrument every program point and record the integer-valued variables. Then AbsDaikon, our extension of Daikon [6] takes these concrete states and then infers constraints over the Interval, Zone, and Octagon templates only, with the rest of Daikon’s default templates disabled.

Inference Policies. In our experiments we first run a soot-based abstract interpreter over Octagon domain and use its results as an Oracle to compare the results from our proposed technique. Since static analyzer computes in Jimple, a 3-address code IR, and the inference is performed over the source code, we use a program that maps Jimple’s variables to source code variables.

Our inference policy is conservative with respect to domain expressiveness. If an inferred source-level variable does not appear in a constraint, it is treated as Unbounded. If the computed reference contains a Jimple-generated temporary variable that is not visible at the source level, the comparison interprets it as Octagon, since this is the most conservative choice. Jimple reference variables such as `r0`, which represent this, are excluded from the comparison. Thus, the policy may over-approximate the needed domain type by recommending a more expressive domain, but in our experiments

Table 2. Confusion matrix for inferred versus computed domain types. Rows are inferred and columns are computed.

Inf \ Comp	O	Z	I	C	τ	Total
Octagon	32	34	40	0	30	136
Zone	0	351	3	4	19	377
Interval	0	0	53	0	13	66
Constant	0	0	0	29	1	30
Unbounded	0	0	0	0	394	394
Total	32	385	96	33	457	1003

it does not recommend a less expressive domain than the computed reference.

This preliminary work evaluates the feasibility of abstract-domain type inference rather than comparing the runtime of an Octagon-based static analyzer with the runtime of our inference pipeline. Since the inference component still in a prototype state we do not report runtimes.

4.2 Results

We answer the following research questions:

RQ1: Do inferred recommendations ever select a less expressive domain than the computed one?

RQ2: How often do inferred recommendations exactly match the computed domain?

RQ3: When inferred domain types are more expressive than the computed, what accounts for the mismatch?

Table 2 summarizes the results, where the diagonal elements are the number of correctly identified domain types, the lower-left triangle contains cases where the inferred domain is less expressive than the computed reference, where an analysis would need a more expressive domain type than what our technique recommended. As the table shows, all those values are zero, therefore, the empirical evaluation confirms the value of our heuristics to err on the safe side. Thus, in our benchmark suite, the inference never recommends a domain that is less expressive than the computed reference.

The diagonal elements are the number of correctly identified domain types, whose sum is 859, or 85.6%. The sum of elements on the top right triangle are ones that inferred at more precise domain than computed. The total number is 144 or 14.4%, which is the accuracy of the approach.

To examine the extent of over-approximation, we evaluated the *domain type precision gap*, which is the distance in the total order of abstract domains between computed and inferred domain types, e.g., the gap between Unbounded to Octagonal is four. Table 3 details the results of this evaluation. The total one for one precision distance is 38; total between two precision gaps is 57; in three precision gaps is 19; and 30 in four. Note, the inferred approach categorized a large number of Zone and Unbounded correctly. Unbounded is quite important since a static analyzer may not even track/perform

Table 3. Cases where the inferred domain is more precise than the computed domain.

Inferred	Computed	Gap	Count
Octagon	Unbounded	4	30
Zone	Unbounded	3	19
Octagon	Interval	2	40
Interval	Unbounded	2	13
Zone	Constant	2	4
Octagon	Zone	1	34
Zone	Interval	1	3
Constant	Unbounded	1	1
Total	—	—	144

computation over that variable and just assign it to the T value. To answer RQ2, the accuracy of our implementation is 85.6%, and 14.5% incorrectly identified, among those about 5% with gap distances of 3 and 4.

To understand why our approach missed 14.5%, we further investigated accuracy mismatches. We found that among 144 over-approximating cases, 72 are due to representation mismatches: static analysis works on Jimple, a three address intermediate representation while inference is performed at the source-code level and cannot track intermediate variables. Per our policy, the intermediate variables of Jimple is safely approximated with Octagonal. Without these cases the accuracy of inferences becomes 92.3%.

The remaining 72 mismatches are due to the Octagonal analysis’s inability to maintain a relation between three unbounded variables, e.g., processing assignment statements like $x = y + z$. For example, if in this case, y and z are unbounded, then the analysis keeps all three variables as unbounded. Moreover, if after this assignment statement there is a conditional statement, such as $\text{if } (x > 0)$, then the analysis computes x to have Interval type, while y and z remain unbounded. However, inference determines that y and z have Octagonal type, because AbsDaikon would infer $y + z > 0$ on the true branch. In cases where y and z are bounded, Octagonal analysis can resolve the relationships between the three variables and determine bounds for the original expression of $x = y + z$. Thus, Octagonal analysis can resolve the correct domain types between the variables x , y , and z . As such, it is in these unbounded cases in which our technique can infer a higher domain categorization than that of static analysis.

5 Related Work

There are two main research directions related to our proposed approach. The first studies how to find sufficient abstractions for static analysis, and the second studies how to infer abstract states from concrete program states.

Approaches for finding sufficient abstractions can be further categorized by how the abstractions are obtained. One

category consists of refinement-based approaches [11, 19]. These approaches typically start with a coarse abstraction and, when a desired program property cannot be proved, they use information from the failed attempt to refine the abstraction. Although our work considers numerical domains rather than context- or object-sensitivity levels and infers them from concrete states, it shares with these approaches the goal of identifying abstractions at fine granularity.

A second category precomputes abstractions using a light-weight program analysis before running the main analysis. The most closely related work is by [Apel et al. \[1\]](#), where the authors statically examine how each variable is used and map variables to suitable abstract-domain types, which are then passed to a model checker. [Demyanova et al. \[5\]](#) similarly use variable roles to generate program-specific predicates and interpolation templates for predicate abstraction. Our approach also identifies domain types for program variables. However, it does so at a finer abstraction context, i.e., variable-location pairs rather than variables alone, and infers the domain types from concrete states rather than from static usage patterns.

The second research direction uses concrete program states to infer abstractions. The pioneering Daikon tool [6] infers likely program invariants from observed concrete states. Our work is inspired by dynamic invariant inference, but differs in both goal and output. We generate only invariants corresponding to numerical abstract-domain templates, and we use those invariants to infer the required domain type rather than to report likely program invariants as analysis results.

The work of [Ho et al. \[8\]](#) is conceptually close to ours because it also constructs abstractions from concrete program states. However, their approach represents abstract states statistically, for example, using clustering parameters, rather than as formulas in a fixed numerical abstract domain. In contrast, our approach explicitly describes concrete states using numerical abstract-domain predicates and then uses those predicates to infer the domain type required at each variable-location pair.

Overall, our work differs from prior abstraction-selection techniques along three dimensions. First, the abstraction choices are standard numerical domains, ordered by expressiveness from Unbounded to Octagons. Second, the abstraction context is a variable-location pair rather than a whole program, query, variable, call site, or allocation site. Third, the selection comes from concrete states rather than static usage patterns or repeated static-analysis feedback.

6 Conclusion and Future Work

In this paper, we proposed a domain-type inference approach that identifies the numerical abstract domain recommended for each variable at each program location. The approach uses MultisetGA to generate a large number of concrete

states, improving the confidence of AbsDaikon when inferring predicates that characterize the shape of the required abstract domain. We then represent the inferred predicates as an Octagonal abstract state and inspect its matrix representation to classify each variable. We empirically evaluated the approach on 20 benchmark programs by comparing the inferred domain types against domain types computed by an Octagon-based static analyzer. The results show that our approach exactly matches the computed reference domain in 85.6% of variable-location pairs, and in the remaining cases recommends a more expressive domain rather than a less expressive one.

Future Work In refining our approach, we would like to improve the efficiency of generating concrete states, so that the combined generation and inference time is justified. We would also like to handle the various cases of inaccuracy such that our inference and static analysis comparisons can operate over the same set of variables such that intermediate variables do not cause issues. Furthermore, we wish to add appropriate handling for unbounded variables such that large discrepancies between the approaches, e.g., large domain type precision gaps, no longer remain. Finally, we wish to validate this approach by integrating the inferred information to a static analyzer such that we can improve the efficiency of analysis without sacrificing overall precision.

Acknowledgments

This work is partly supported by the National Science Foundation under Grant No. CCF-19-42044. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

References

- [1] Sven Apel, Dirk Beyer, Karlheinz Friedberger, Franco Raimondi, and Alexander von Rhein. 2013. Domain Types: Abstract-Domain Selection Based on Variable Usage. In *Hardware and Software: Verification and Testing*, Valeria Bertacco and Axel Legay (Eds.). Springer International Publishing, Cham, 262–278. doi:10.1007/978-3-319-03077-7_18
- [2] Sahar Badihi, Yi Li, and Julia Rubin. 2021. EqBench: A Dataset of Equivalent and Non-equivalent Program Pairs. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 610–614. doi:10.1109/MSR52588.2021.00084
- [3] Kenny Ballou and Elena Sherman. 2026. Minimal Comparison of Octagonal Abstract Domains. In *Proceedings of the 11th ACM SIGPLAN International Workshop on Numerical and Symbolic Abstract Domains (NSAD '26)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3840563.3843710
- [4] Patrick Cousot and Nicolas Halbwachs. 1978. Automatic Discovery of Linear Constraints Among Variables of a Program. In *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '78)*. ACM, New York, NY, USA, 84–96. doi:10.1145/512760.512770
- [5] Yulia Demyanova, Philipp Rümmer, and Florian Zuleger. 2017. Systematic Predicate Abstraction Using Variable Roles. In *NASA Formal Methods - 9th International Symposium, NFM 2017 (Lecture Notes in Computer Science, Vol. 10227)*. Springer, 265–281. doi:10.1007/978-3-319-57288-8_18
- [6] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. 2007. The Daikon System for Dynamic Detection of Likely Invariants. *Sci. Comput. Program.* 69, 1-3 (Dec. 2007), 35–45. doi:10.1016/j.scico.2007.01.015
- [7] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (Szeged, Hungary) (ESEC/FSE '11)*. Association for Computing Machinery, New York, NY, USA, 416–419. doi:10.1145/2025113.2025179
- [8] Vivian M. Ho, Chris Alvin, Jimmie D. Lawson, Supratik Mukhopadhyay, and Brian Peterson. 2023. Program analysis using empirical abstraction. *Int. J. Softw. Tools Technol. Transf.* 25, 4 (July 2023), 431–452. doi:10.1007/s10009-023-00709-y
- [9] Gunel Jahangirova, David Clark, Mark Harman, and Paolo Tonella. 2018. OASIs: oracle assessment and improvement tool. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (Amsterdam, Netherlands) (ISSTA 2018)*. Association for Computing Machinery, New York, NY, USA, 368–371. doi:10.1145/3213846.3229503
- [10] Kim G. Larsen, Fredrik Larsson, Paul Pettersson, and Wang Yi. 2003. Compact Data Structures and State-Space Reduction for Model Checking Real-Time Systems. *Real-Time Systems* 25, 2/3 (2003), 255–275. doi:10.1023/a:1025132427497
- [11] Percy Liang, Omer Tripp, and Mayur Naik. 2011. Learning minimal abstractions. *SIGPLAN Not.* 46, 1 (Jan. 2011), 31–42. doi:10.1145/1925844.1926391
- [12] Antoine Miné. 2002. A Few Graph-Based Relational Numerical Abstract Domains. *Lecture Notes in Computer Science* (2002), 117–132. doi:10.1007/3-540-45789-5_11
- [13] Antoine Miné. 2006. The Octagon Abstract Domain. *Higher-Order and Symbolic Computation* 19, 1 (2006), 31–100. doi:10.1007/s10990-006-8609-1
- [14] Teddy Moore. 2026. MultisetGA: Diversity-Driven Test Generation for Invariant Inference. GitLab repository. <https://gitlab.com/anonrepo123-group/anonrepo123-project>
- [15] Cu D. Nguyen, Alessandro Marchetto, and Paolo Tonella. 2013. Automated oracles: an empirical study on cost and effectiveness. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (Saint Petersburg, Russia) (ESEC/FSE 2013)*. Association for Computing Machinery, New York, NY, USA, 136–146. doi:10.1145/2491411.2491434
- [16] Thanhvu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. 2014. DIG: A Dynamic Invariant Generator for Polynomial and Array Invariants. *ACM Trans. Softw. Eng. Methodol.* 23, 4, Article 30 (Sept. 2014), 30 pages. doi:10.1145/2556782
- [17] Matt Staats, Shin Hong, Moonzoo Kim, and Gregg Rothermel. 2012. Understanding user understanding: determining correctness of generated program invariants. In *Proceedings of the 2012 International Symposium on Software Testing and Analysis (Minneapolis, MN, USA) (ISSTA 2012)*. Association for Computing Machinery, New York, NY, USA, 188–198. doi:10.1145/2338965.2336776
- [18] Xinyu Wang, Greg Anderson, Isil Dillig, and K. L. McMillan. 2018. Learning Abstractions for Program Synthesis. In *Computer Aided Verification*, Hana Chockler and Georg Weissenbacher (Eds.). Springer International Publishing, Cham, 407–426. doi:10.1007/978-3-319-96145-3_22
- [19] Xin Zhang, Mayur Naik, and Hongseok Yang. 2013. Finding optimum abstractions in parametric dataflow analysis. *SIGPLAN Not.* 48, 6 (June 2013), 365–376. doi:10.1145/2499370.2462185

Received 2026-06-21; accepted 2026-08-04